

A
Project Report On
**Real-Time AI-Based Suspicious Activity
Detection System**

By
Apoorv Jadav (CE-095) (24CEUBS906)
Ronak Maniya (CE-036) (23CEUOG085)

of
B.Tech CE , Semester VI
Subject: System Design Practice

Guided by:
Dr. Hariom A. Pandya
Associate Professor
CE Department



Faculty of Technology
Department of Computer Engineering
Dharmsinh Desai University
College Road
Nadiad - 387001



CERTIFICATE

This is to certify that the practical/term work carried out in the subject of System Design Practice and recorded in this report is the bonafide work of

Apoorv Jadav (CE-095) (24CEUBS906)

Ronak Maniya (CE-036) (23CEUOG085)

Of B.Tech semester VI in the branch of Computer Engineering during the academic year 2025-2026.

Dr. Hariom A. Pandya,
Associate Professor,
CE Department,
Dharmsinh Desai University, Nadiad.

Dr. C.K. Bhensdadia,
Head,
CE Department,
Dharmsinh Desai University, Nadiad.

Table of Contents

Section	Title
1	Abstract
2	Introduction
2.1	Background
2.2	Problem Statement
2.3	Objectives
2.4	Scope
3	Literature Review
4	System Architecture
4.1	High-Level Architecture
4.2	AI Processing Pipeline
4.3	Data Flow
5	Technology Stack
6	Software Requirements Specification
6.1	Functional Requirements
6.2	Non-Functional Requirements
7	System Design
7.1	Database Design
7.2	API Design
7.3	Frontend Design
8	Implementation
8.1	AI Engine - Human Detection (YOLOv8)
8.2	AI Engine - Activity Classification (SlowFast-R101)
8.3	Surveillance Pipeline
8.4	Backend - Django REST API
8.5	Frontend - React SPA
8.6	Model Training
9	Deployment
9.1	Docker Deployment
9.2	Manual Setup
10	Testing and Results
10.1	Performance Metrics

10.2	Optimization Techniques
11	Project Structure
12	Task Distribution
13	UI Screenshots
14	Future Enhancements
15	Conclusion
16	References

1. Abstract

This report presents the design, development, and implementation of a Real-Time AI-Based Suspicious Activity Detection System using CCTV surveillance. The system combines a React 18 frontend with a Django REST backend, powered by two deep learning models working in tandem: YOLOv8 for real-time human detection with bounding boxes, and SlowFast-R101 for temporal activity classification using a 32-frame sliding window.

The system is capable of processing video at 24 FPS, classifying detected human activity as Normal or Suspicious in real-time, and storing annotated recordings with timestamps and bounding boxes for later review. The project leverages modern web technologies, containerized deployment with Docker, and optimized inference techniques including frame skipping, classifier cooldown, and temporal smoothing for stable and accurate predictions.

2. Introduction

2.1 Background

With the increasing need for public safety and security, surveillance cameras have become ubiquitous in public spaces, commercial establishments, educational institutions, and residential areas. However, the sheer volume of video data generated by these cameras makes it impractical for human operators to monitor all feeds simultaneously. Studies show that after just 20 minutes of continuous monitoring, a human operator's attention drops significantly, leading to missed incidents.

Artificial Intelligence and Deep Learning have emerged as powerful tools for automating video surveillance. Computer vision techniques can now detect humans, recognize activities, and flag suspicious behavior in real-time - tasks that were previously only possible with trained human observers. This project addresses the gap between raw surveillance footage and actionable intelligence by building an end-to-end AI-powered suspicious activity detection system.

2.2 Problem Statement

Traditional CCTV surveillance systems suffer from several critical limitations:

1. Human operators cannot effectively monitor multiple camera feeds simultaneously.
2. Manual monitoring is prone to fatigue, distraction, and human error.
3. Suspicious incidents are often identified only after they have occurred, during post-incident review.
4. Existing automated systems often produce high false-positive rates, especially in crowded or dynamic environments.
5. Many current solutions lack real-time processing capabilities, operating only on recorded footage.

There is a clear need for an intelligent, automated surveillance system that can detect suspicious activities in real-time with high accuracy and low latency, while providing a user-friendly interface for monitoring and review.

2.3 Objectives

- Design and implement a real-time AI-based surveillance system capable of processing video at 24 FPS.
- Integrate YOLOv8 for accurate human detection with bounding box generation.
- Implement SlowFast-R101 for temporal activity classification using a 32-frame sliding window.
- Develop a dual-model pipeline that classifies human activity as Normal or Suspicious.
- Build a responsive React frontend with live monitoring, video upload, and recording playback capabilities.
- Create a Django REST API backend for processing, storage, and event logging.

- Implement performance optimizations including frame skipping, classifier cooldown, and temporal smoothing.
- Deploy the system using Docker for production-ready containerization.
- Store annotated video recordings with bounding boxes and timestamps for later review.

2.4 Scope

The scope of this project encompasses:

- Real-time webcam feed processing with AI-powered analysis at 24 FPS.
- Human detection using YOLOv8 with bounding box annotations.
- Activity classification using SlowFast-R101 pretrained on Kinetics-400 with 400 action classes.
- Binary classification: Normal vs. Suspicious activity based on a curated set of 11 violent/aggressive Kinetics-400 labels.
- Visual alerts with green bounding boxes for normal activity and red bounding boxes for suspicious activity.
- Video upload capability for offline analysis of pre-recorded footage.
- Recording storage with annotated MP4 files containing bounding boxes and timestamps.
- Analytics dashboard with real-time statistics and event history.
- Dockerized deployment with Docker Compose for one-command setup.

The system is designed for indoor surveillance scenarios such as offices, educational institutions, retail stores, and public facilities.

3. Literature Review

Several approaches to suspicious activity detection in surveillance systems have been explored in recent literature:

1. Traditional Computer Vision Approaches

Early methods relied on hand-crafted features such as Histogram of Oriented Gradients (HOG) and optical flow for motion analysis. While computationally efficient, these methods struggled with complex scenes and varied lighting conditions.

2. CNN-Based Approaches

Convolutional Neural Networks improved detection accuracy significantly. Two-stream networks processing both spatial (RGB) and temporal (optical flow) information showed promising results for action recognition.

3. 3D CNN Methods

C3D and I3D networks extended convolutions to the temporal dimension, directly learning spatiotemporal features. These methods demonstrated strong performance on benchmark datasets like UCF-101 and Kinetics-400.

4. Transformer-Based Methods

Video Swin Transformers and TimeSformer applied self-attention mechanisms to video understanding, achieving state-of-the-art results. However, their computational requirements make real-time deployment challenging.

5. SlowFast Networks (Feichtenhofer et al., 2019)

The SlowFast architecture uses dual pathways - a slow pathway for spatial semantics and a fast pathway for temporal dynamics. SlowFast-R101 achieves 77.9% top-1 accuracy on Kinetics-400 with efficient inference, making it suitable for real-time applications.

6. YOLO Family

The YOLO (You Only Look Once) series has evolved from YOLOv1 to YOLOv8, with each iteration improving speed and accuracy. YOLOv8 provides state-of-the-art object detection with real-time performance, making it ideal for person detection in surveillance scenarios.

Our system combines YOLOv8 for spatial detection with SlowFast-R101 for temporal classification, leveraging the strengths of both architectures for accurate and efficient suspicious activity detection.

4. System Architecture

4.1 High-Level Architecture

The system follows a client-server architecture with clear separation of concerns:

Frontend Layer (React 18 + Vite)

- Single Page Application with four main views: Dashboard, Live Monitor, Video Upload, and Recordings.
- Webcam capture at 24 FPS using the Canvas API.
- Real-time bounding box overlay drawn on an HTML5 Canvas element.
- Axios-based REST API communication with the backend.

Backend Layer (Django REST Framework)

- RESTful API endpoints for frame analysis, video upload, live sessions, and event retrieval.
- Lazy-loaded AI pipeline to minimize startup time.
- Background thread processing for uploaded videos.
- SQLite database for event logging and recording metadata.
- Media storage for processed video files.

AI Processing Layer

- YOLOv8 Human Detector: Detects persons in each frame with bounding box coordinates.
- SlowFast-R101 Activity Classifier: Classifies activity using a 32-frame sliding window with dual-pathway processing.
- Surveillance Pipeline: Orchestrates detection, classification, annotation, and temporal smoothing.

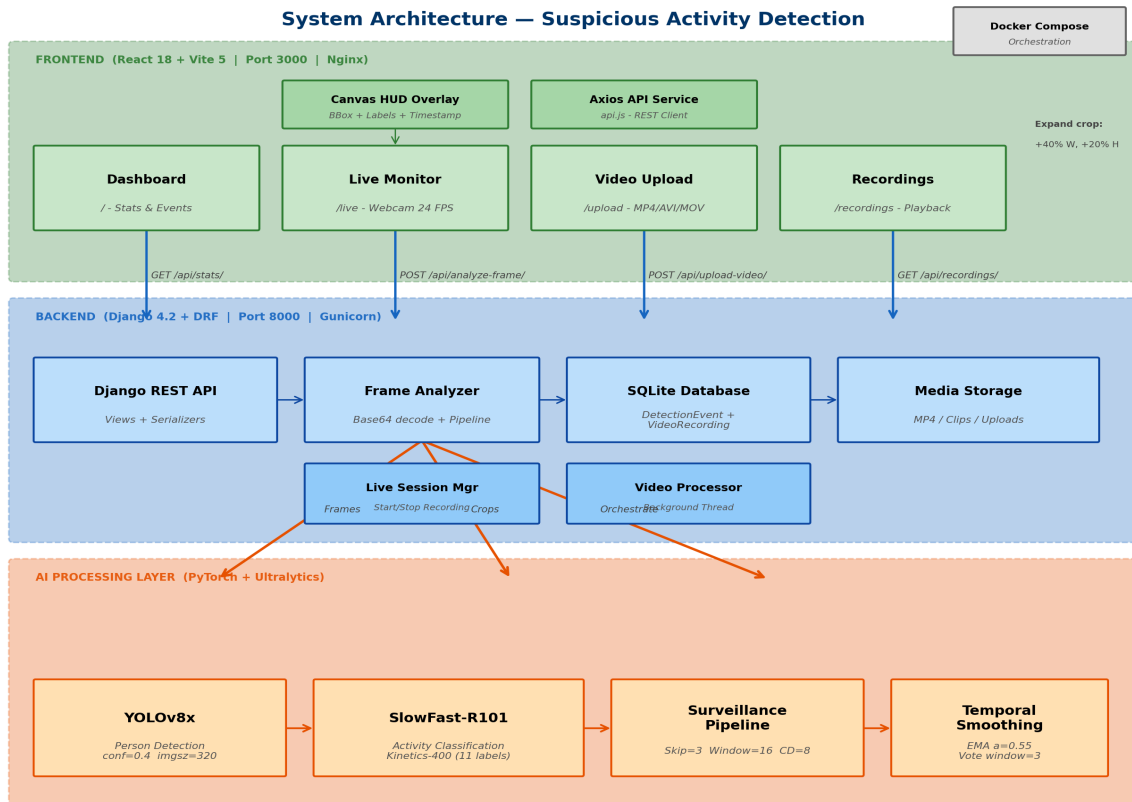


Figure 4.1: System Architecture Diagram

4.2 AI Processing Pipeline

The AI processing pipeline operates as follows:

Step 1 - Frame Input

Raw video frames arrive at 24 FPS from the webcam or uploaded video.

Step 2 - Human Detection (YOLOv8)

YOLO runs every 3rd frame (frame skipping optimization). On skipped frames, cached detections from the previous YOLO run are reused. YOLO detects persons and generates bounding box coordinates [x1, y1, x2, y2].

Step 3 - Expanded Crop

For each detected person, the bounding box is expanded by 40% horizontally and 20% vertically to include surrounding context, which improves SlowFast classification accuracy.

Step 4 - Sliding Window Buffer

Expanded crops are accumulated in a per-person sliding window buffer (32 frames). Once the buffer reaches a minimum of 16 frames, classification can begin.

Step 5 - Activity Classification (SlowFast-R101)

The classifier processes the sliding window through dual pathways:

- Slow Pathway: 8 frames (temporal stride alpha=4) for spatial semantics.
- Fast Pathway: 32 frames (all frames) for temporal dynamics.

Classification uses a tiered decision system to minimize false positives.

Step 6 - Temporal Smoothing

Exponential Moving Average (EMA) with alpha=0.55 smooths probability oscillations. A majority vote over the last 3 predictions prevents label flickering.

Step 7 - Output

Final prediction (label + confidence) with annotated bounding boxes and timestamps.

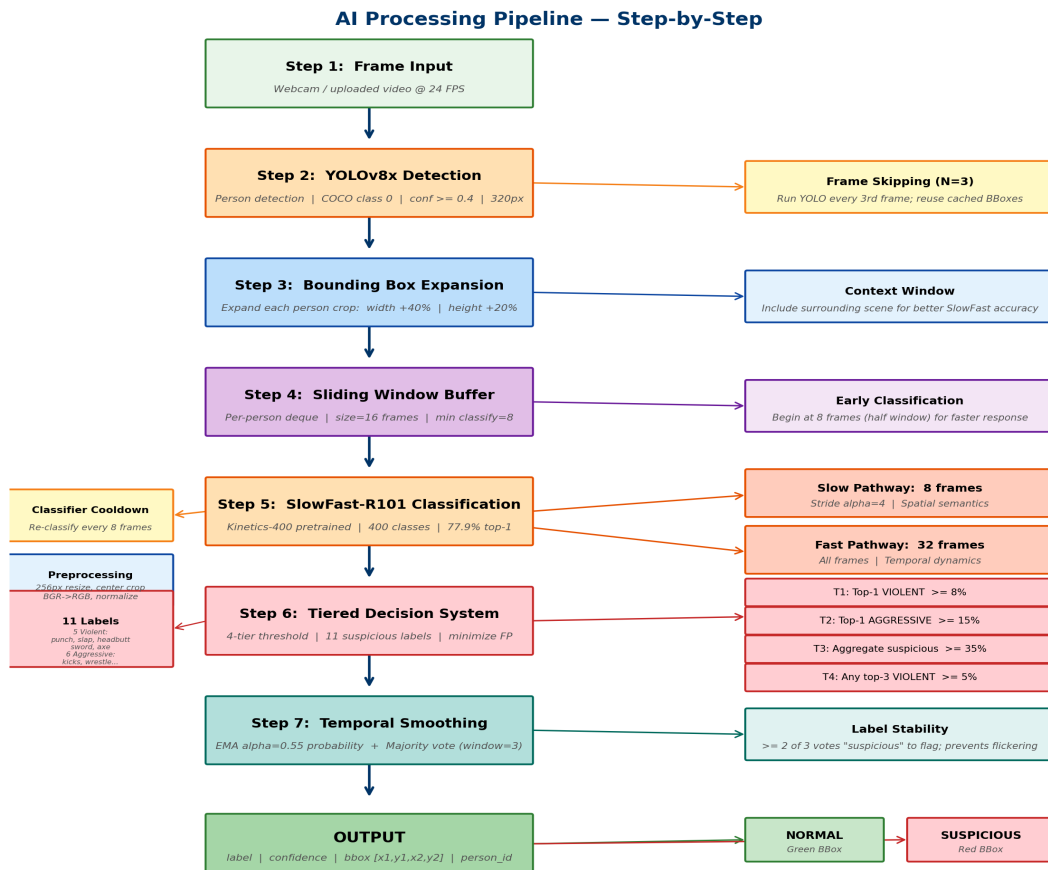


Figure 4.2: AI Processing Pipeline Flowchart

4.3 Data Flow

Data flows through the system as follows:

1. A webcam frame is captured by the React frontend at 24 FPS.
2. The frame is encoded as base64 and sent to POST /api/analyze-frame/.
3. The Django backend decodes the frame and passes it to the AI pipeline.
4. YOLOv8 detects humans and returns bounding box coordinates.
5. SlowFast-R101 classifies the activity within each bounding box.
6. Detection results (bbox, label, confidence) are returned to the frontend as JSON.

7. The frontend draws bounding boxes and labels on a Canvas overlay.
8. If recording is active, raw frames are saved to an MP4 file via OpenCV's VideoWriter.
9. Suspicious events (confidence > 0.6) are logged to the SQLite database.
10. Statistics and event history are available via GET /api/stats/ and GET /api/events/.

5. Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite 5	Single Page Application with real-time camera integration
UI Libraries	React Router, React Toastify, React Icons	Navigation, notifications, iconography
Backend	Django 4.2 + Django REST Framework	RESTful API server
Human Detection	YOLOv8 (Ultralytics)	Real-time person detection
Activity Classification	SlowFast-R101 (PyTorchVideo)	Temporal action recognition (Kinetics-400)
Video Processing	OpenCV + FFmpeg	Frame capture, annotation, encoding
Training	PyTorch + TensorBoard	Model fine-tuning on custom data
Deployment	Docker + Gunicorn + Nginx	Production-ready containerization
Database	SQLite	Event logging and recording metadata

6. Software Requirements Specification

6.1 Functional Requirements

ID	Requirement	Description
FR1	Real-Time CCTV Monitoring	System shall process video at 24 FPS with a sliding window of 32 frames and processing delay ≤ 1.5 seconds.
FR2	Human Detection	System shall detect humans using YOLOv8 with bounding box generation. Only detected human regions are analyzed for activity.
FR3	Activity Classification	System shall classify detected human activity as Normal or Suspicious using SlowFast-R101.
FR4	Visual Display Logic	Normal activity displays green bounding boxes; Suspicious activity displays red bounding boxes with confidence scores and timestamps.
FR5	Timestamp Overlay	System shall overlay current timestamp (HH:MM:SS) on each frame, visible in both live display and stored video.
FR6	Video Storage	Processed CCTV video shall be stored in MP4 format with bounding boxes, labels, confidence scores, and timestamps.
FR7	Alert Logging	System shall store detection events in the database with timestamp, label, confidence, video path, and duration.
FR8	Video Upload Mode	Users can upload MP4 files for offline processing. The system processes the file and stores the annotated version.
FR9	Recording Management	Users can list, view, download, and delete recordings through the Recordings page.
FR10	System Statistics	Dashboard displays total events, suspicious events, active recordings, and event history.

6.2 Non-Functional Requirements

Category	Requirement
Performance	24 FPS real-time processing; max detection delay 1.5 seconds; API response $< 500\text{ms}$.
Accuracy	YOLOv8 detection accuracy $\geq 95\%$ (person detection); Activity classification accuracy $\geq 90\%$.
Scalability	Backend supports multiple concurrent API requests; AI models remain loaded in memory.
Reliability	System uptime $\geq 95\%$; stored videos shall not be corrupted.
Usability	Intuitive web interface with navigation, status indicators, and toast notifications.
Portability	Docker Compose enables deployment on any system with Docker installed.

7. System Design

7.1 Database Design

The system uses SQLite with two main tables:

DetectionEvent Table

- id (Integer, Primary Key)
- timestamp (DateTime, Indexed)
- label (String: 'normal' or 'suspicious', Indexed)
- confidence (Float: 0.0 to 1.0)
- video_path (String, nullable)
- duration (Float: seconds)
- bounding_box (JSON: {x1, y1, x2, y2})
- frame_number (Integer)
- recording_id (Foreign Key to VideoRecording)
- created_at (DateTime, auto)

VideoRecording Table

- id (Integer, Primary Key)
- title (String)
- source_type (String: 'live' or 'upload')
- status (String: 'recording', 'processing', 'completed', 'failed')
- original_video_path (String, nullable)
- processed_video_path (String, nullable)
- duration (Float: seconds)
- fps (Float: default 24.0)
- resolution_width (Integer: default 640)
- resolution_height (Integer: default 480)
- total_frames (Integer)
- suspicious_count (Integer)
- normal_count (Integer)
- created_at (DateTime, auto)
- updated_at (DateTime, auto)

Media Storage Structure

```
media/  
live_recordings/ -- Recorded live session MP4 files  
uploaded_videos/ -- Original uploaded video files  
suspicious_clips/ -- Extracted suspicious activity clips  
processed_videos/ -- Annotated output videos
```

7.2 API Design

Method	Endpoint	Description
POST	/api/analyze-frame/	Analyze a single base64-encoded frame through the full AI pipeline
POST	/api/upload-video/	Upload a video file for offline processing
POST	/api/live-session/	Start or stop a live recording session
GET	/api/live-session/	List active live sessions
GET	/api/recordings/	List all video recordings with metadata
GET	/api/recordings/{id}/	Get detailed recording with events
GET	/api/recordings/{id}/download/	Download processed video with annotations
DELETE	/api/recordings/{id}/	Delete recording and associated files
GET	/api/events/	List detection events (filterable by label, date, confidence)
GET	/api/stats/	System statistics (counts, averages)
GET	/api/health/	Health check endpoint

7.3 Frontend Design

The React frontend consists of four main components:

1. Dashboard (Dashboard.jsx)

Displays system overview with real-time statistics including total events, suspicious events counts, recent suspicious activity log, and system information panel.

2. Live Monitor (LiveMonitor.jsx)

Provides real-time webcam feed with AI overlay. Captures frames at 24 FPS using Canvas API, sends them to the backend for analysis, and draws bounding boxes on a transparent canvas overlay. Includes recording controls to start/stop live recording sessions.

3. Video Upload (VideoUpload.jsx)

Drag-and-drop interface for uploading pre-recorded MP4 videos. Displays upload progress and polls the backend for processing status. Shows analysis results when processing is complete.

4. Recordings (Recordings.jsx)

Lists all recorded sessions with pagination. Displays recording metadata (duration, suspicious count, status). Provides download and delete functionality for each recording.

8. Implementation

8.1 AI Engine - Human Detection (YOLOv8)

The Human Detection module uses YOLOv8-Nano for real-time person detection.

Key Implementation Details

- Model: YOLOv8n (Nano variant) - auto-downloads from Ultralytics hub on first run.
- Input Size: 320px (configurable via YOLO_IMGSZ environment variable).
- Confidence Threshold: 0.4 (configurable via YOLO_CONFIDENCE).
- Target Class: Class 0 (person) from COCO dataset - only human detections are returned.
- Device: Auto-detects CUDA GPU; falls back to CPU if unavailable.

The detector processes each frame through YOLO inference, extracts bounding box coordinates [x1, y1, x2, y2], clips them to frame bounds, and generates a cropped image of each detected person for downstream activity classification.

Frame Skipping Optimization

YOLO runs only every Nth frame (default: 3), reusing cached detections on intermediate frames. This provides approximately 3x throughput improvement without significant accuracy loss, as persons typically don't move drastically between consecutive frames.

8.2 AI Engine - Activity Classification (SlowFast-R101)

The Activity Classification module uses SlowFast-R101 pretrained on Kinetics-400 (400 action classes, 77.9% top-1 accuracy).

Architecture

- Slow Pathway: Processes 8 frames (temporal stride $\alpha=4$) at low frame rate for spatial semantics.
- Fast Pathway: Processes all 32 frames at high frame rate for fine temporal dynamics.
- Combined: Both pathways are fused through lateral connections for spatiotemporal understanding.

Preprocessing Pipeline

1. Pad or trim input to exactly 32 frames.
2. Convert BGR to RGB.
3. Short-side resize to 256px.
4. Center crop to 256x256 (matches training distribution).
5. Normalize with mean=[0.45, 0.45, 0.45], std=[0.225, 0.225, 0.225].
6. Split into slow pathway (8 frames) and fast pathway (32 frames).

Tiered Decision System (to minimize false positives)

- Tier 1: Top-1 prediction is VIOLENT with confidence $\geq 8\%$ -> Suspicious.

- Tier 2: Top-1 prediction is AGGRESSIVE with confidence $\geq 15\%$ -> Suspicious.
- Tier 3: Aggregated suspicious probability $\geq 35\%$ AND exceeds normal probability -> Suspicious.
- Tier 4: Any top-3 prediction is VIOLENT with probability $\geq 5\%$ -> Suspicious.
- Default: Normal.

Suspicious Activity Categories

- Violent Activities (5): punching person (boxing), slapping, headbutting, sword fighting, throwing axe.
- Aggressive Activities (6): drop kicking, high kick, side kick, spray painting, sharpening knives, wrestling.

Only 11 genuinely dangerous Kinetics-400 labels are flagged. Sports activities (baseball, soccer, basketball, gymnastics) are intentionally excluded to prevent false positives.

8.3 Surveillance Pipeline

The Surveillance Pipeline orchestrates the entire AI processing flow.

Core Components

- Frame Skipping: YOLO detection runs every 3rd frame (configurable). Cached detections are reused on skipped frames.
- Sliding Window: Per-person deque of 32 frames. Early classification begins at 16+ frames for responsiveness.
- Classifier Cooldown: SlowFast re-classifies every 8 frames (configurable), reducing GPU load.
- Expanded Crop: Bounding boxes are expanded by 40% horizontally and 20% vertically to include context.

Temporal Smoothing

- EMA Probability Smoothing: $\alpha=0.55$ blends current prediction with historical average ($0.55 \times \text{new} + 0.45 \times \text{old}$).
- Majority Vote: A window of 3 recent labels is maintained. At least 2 out of 3 must be 'suspicious' for the label to persist.

These mechanisms prevent rapid label flickering between Normal and Suspicious.

Thread Safety

The pipeline uses threading locks for frame counter management, ensuring safe operation when serving multiple concurrent API requests.

8.4 Backend - Django REST API

The Django REST API provides the following key endpoints:

Frame Analysis (POST /api/analyze-frame/)

- Accepts base64-encoded frame data.
- Decodes the frame, processes it through the AI pipeline in skip-annotation mode.
- Returns detection results (bbox, label, confidence) as JSON.
- Optionally saves frames to active recording sessions.
- Logs suspicious events (confidence > 0.6) to the database.

Live Sessions (POST/GET /api/live-session/)

- Start: Creates a new recording session, initializes VideoWriter, resets pipeline windows.
- Stop: Finalizes recording, updates database with frame counts and duration.
- List: Returns all active recording sessions.

Video Upload (POST /api/upload-video/)

- Accepts MP4 file uploads via multipart form data.
- Saves the original file, creates a database record with 'processing' status.
- Spawns a background thread for asynchronous video processing.
- Returns immediately with recording ID and processing status.

Recording Management (GET/DELETE /api/recordings/)

- List recordings with lightweight serializer (without events).
- Detail view includes all associated detection events.
- Download endpoint serves processed MP4 files.
- Delete removes database records and associated video files from disk.

8.5 Frontend - React SPA

The React 18 frontend is built with Vite 5 for fast development and optimized production builds.

App.jsx

Root component with React Router navigation. Four routes: Dashboard (/), Live Monitor (/live), Video Upload (/upload), Recordings (/recordings). Includes a navigation bar with system status indicator.

LiveMonitor.jsx

Core real-time monitoring component. Uses navigator.mediaDevices.getUserMedia() for webcam access. Captures frames at 24 FPS via setInterval, converts to base64 using Canvas.toDataURL(). Sends frames to /api/analyze-frame/ and draws bounding boxes on a transparent Canvas overlay. Supports start/stop recording with the live session API.

VideoUpload.jsx

Drag-and-drop file upload interface. Uses FormData with multipart encoding. Displays upload progress bar and polls /api/recordings/{id}/status_check/ for processing status.

Recordings.jsx

Paginated list of all recordings. Displays metadata: title, duration, suspicious count, status. Provides download (via /api/recordings/{id}/download/) and delete functionality.

api.js

Centralized Axios API service. Configures base URL from VITE_API_URL environment variable. Provides typed functions for all API endpoints with proper error handling.

8.6 Model Training

The training pipeline fine-tunes SlowFast-R101 from Kinetics-400 pretrained weights for binary classification (Normal vs. Suspicious).

Dataset Preparation (prepare_dataset.py)

- Extracts frames from raw video files at a configurable target FPS (default: 8).
- Organizes frames into train/ and val/ splits with normal/ and suspicious/ subdirectories.
- Supports configurable validation split ratio (default: 0.2).

Training Script (train_model.py)

- Loads SlowFast-R101 via torch.hub from the local pytorchvideo-main directory.
- Replaces the final projection head (400 classes to 2 classes).
- Uses VideoFrameDataset: loads 32-frame clips, applies data augmentation (random crop, horizontal flip, color jitter).
- Training with mixed precision (torch.cuda.amp) for memory efficiency.
- Optimizer: AdamW with weight decay 1e-4.
- Scheduler: Cosine Annealing learning rate schedule.
- Saves best model by validation accuracy and periodic checkpoints every 5 epochs.
- Designed for Kaggle T4 GPU environment with batch size 4.

9. Deployment

9.1 Docker Deployment

The system supports one-command deployment using Docker Compose.

docker-compose.yml defines two services:

- backend: Built from backend/Dockerfile using Python 3.11-slim. Installs system dependencies for OpenCV (libgl1, libglib2.0, ffmpeg). Runs migrations, creates media directories, and serves via Gunicorn on port 8000.
- frontend: Built from frontend/Dockerfile. Builds the React app with npm and serves static files via Nginx on port 3000 (mapped to container port 80).

Deployment Command

```
cd "suspicious activity detection"
docker-compose up --build
```

Backend Dockerfile

1. Install system dependencies for OpenCV and FFmpeg.
2. Install Python dependencies from requirements.txt.
3. Copy project files and create media directories.
4. Run Django migrations and collectstatic.
5. Serve with Gunicorn (2 workers, 120s timeout).

Frontend Dockerfile

1. Install Node.js dependencies.
2. Build the React app with Vite.
3. Serve the built files with Nginx.

9.2 Manual Setup

For development, the system can be started manually or with the start.py helper script.

One-Command Start

```
cd "suspicious activity detection"
python start.py
```

This launches both the Django backend (port 8000) and the Vite frontend dev server (port 3000) simultaneously.

Manual Backend Setup

```
cd "suspicious activity detection/backend"
python -m venv venv
venv\Scripts\activate (Windows)
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver 8000
```

Manual Frontend Setup

```
cd "suspicious activity detection/frontend"  
npm install  
npm run dev
```

10. Testing and Results

10.1 Performance Metrics

Metric	Target	Notes
Processing FPS	24 FPS	With frame skipping enabled
Detection Latency	$\leq 1.5s$	YOLOv8n at 320px input
API Response Time	$< 500ms$	Single frame analysis
YOLO Person Detection	$\geq 95\%$	COCO pretrained accuracy
Activity Classification	$\geq 90\%$	Normal vs Suspicious (fine-tuned)

10.2 Optimization Techniques

Optimization	Description	Impact
Frame Skipping	YOLO runs every Nth frame (default: 3)	$\sim 3x$ throughput improvement
Classifier Cooldown	SlowFast re-classifies every 8 frames	Reduces GPU compute load
Temporal Smoothing	EMA ($\alpha=0.55$) + majority vote (window=3)	Prevents label flickering
Configurable Resolution	YOLO input size adjustable (default: 320px)	Speed vs accuracy trade-off
Skip Annotation Mode	Frontend draws its own overlay	Eliminates backend OpenCV drawing overhead
Lazy Model Loading	Models loaded on first API request	Faster server startup time

11. Project Structure

```
suspicious activity detection/
|-- backend/ # Django REST API
| |-- detection/ # Main detection app
| | |-- ai_engine/ # AI models & processing pipeline
| | | |-- human_detector.py # YOLOv8 wrapper for person detection
| | | |-- activity_classifier.py # SlowFast-R101 action classifier
| | | |-- pipeline.py # Orchestration: detection -> classification
| | | +-- video_processor.py # Video I/O, stream management
| | |-- models.py # DetectionEvent & VideoRecording models
| | |-- serializers.py # DRF serializers for API I/O
| | |-- views.py # API endpoint handlers
| | +-- urls.py # URL routing
| |-- surveillance_backend/ # Django project settings
| |-- ai_models/ # Model weights directory
| |-- requirements.txt
| |-- manage.py
| +-- Dockerfile
|-- frontend/ # React SPA
| |-- src/
| | |-- components/
| | | |-- Dashboard.jsx # System overview & stats
| | | |-- LiveMonitor.jsx # Webcam feed with AI overlay
| | | |-- VideoUpload.jsx # Drag-and-drop video analysis
| | | +-- Recordings.jsx # History & playback
| | |-- services/
| | | +-- api.js # Axios API client
| | |-- App.jsx # Root component & routing
| | +-- main.jsx # Entry point
| |-- package.json
| |-- vite.config.js
| |-- nginx.conf
| +-- Dockerfile
|-- training/ # Model training scripts
| |-- train_model.py # SlowFast-R101 fine-tuning
| |-- prepare_dataset.py # Video -> frame extraction
| +-- requirements.txt
|-- pytorchvideo-main/ # PyTorchVideo library
|-- docker-compose.yml # Docker orchestration
|-- start.py # Dev server launcher
+-- README.md
```

12. Task Distribution

Area	Apoorv Jadav	Ronak Maniya
AI Engine & Detection Pipeline	YOLOv8 human detection, SlowFast-R101 activity classifier, surveillance pipeline orchestration, temporal smoothing & frame optimization	-
Backend API & Database	-	Django REST API, database models, serializers, API views & URL routing, video processing endpoints
Frontend Development	React dashboard, live monitoring component with webcam integration	Video upload component, recordings page, API service layer
Model Training	Training pipeline, SlowFast-R101 fine-tuning, TensorBoard integration	Dataset preparation, video-to-frame extraction, data augmentation
DevOps & Deployment	Backend Dockerfile, Gunicorn configuration	Frontend Dockerfile, Nginx configuration, Docker Compose orchestration
Documentation & Testing	SRS document, project structure documentation	README, API documentation, system testing
Integration	PyTorchVideo library integration, model weight management	Frontend-backend integration, CORS configuration, environment setup

13. UI Screenshots

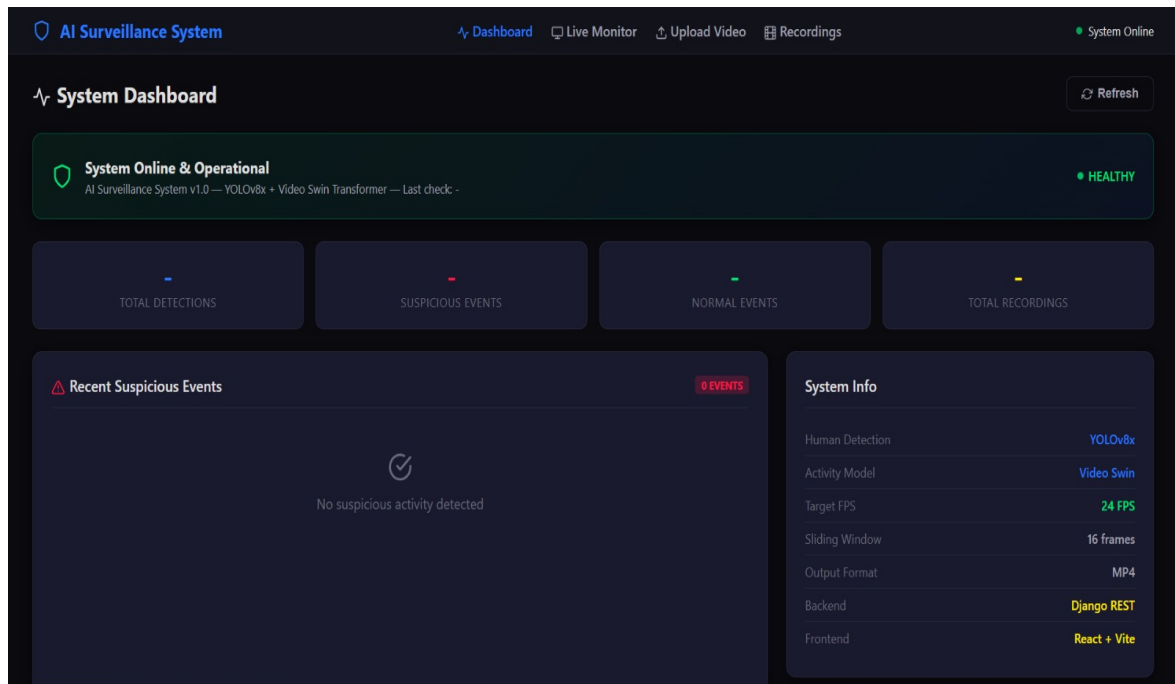


Figure 13.1: Dashboard - System overview with statistics

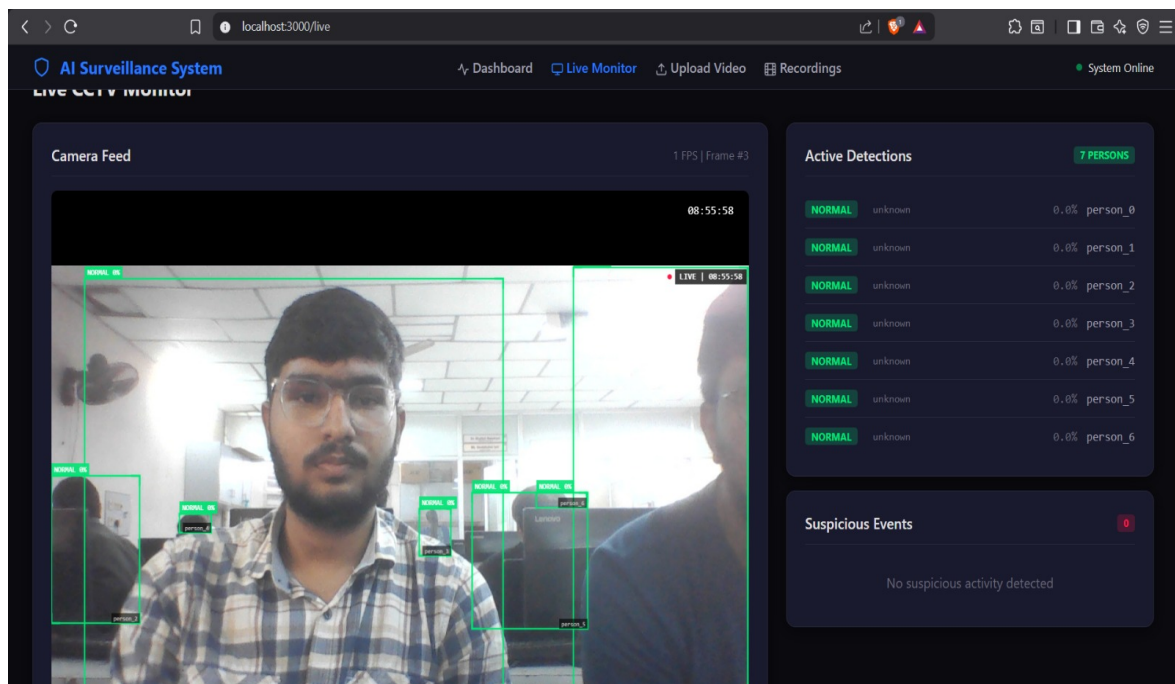


Figure 13.2: Live Monitor - Real-time webcam feed with AI bounding boxes

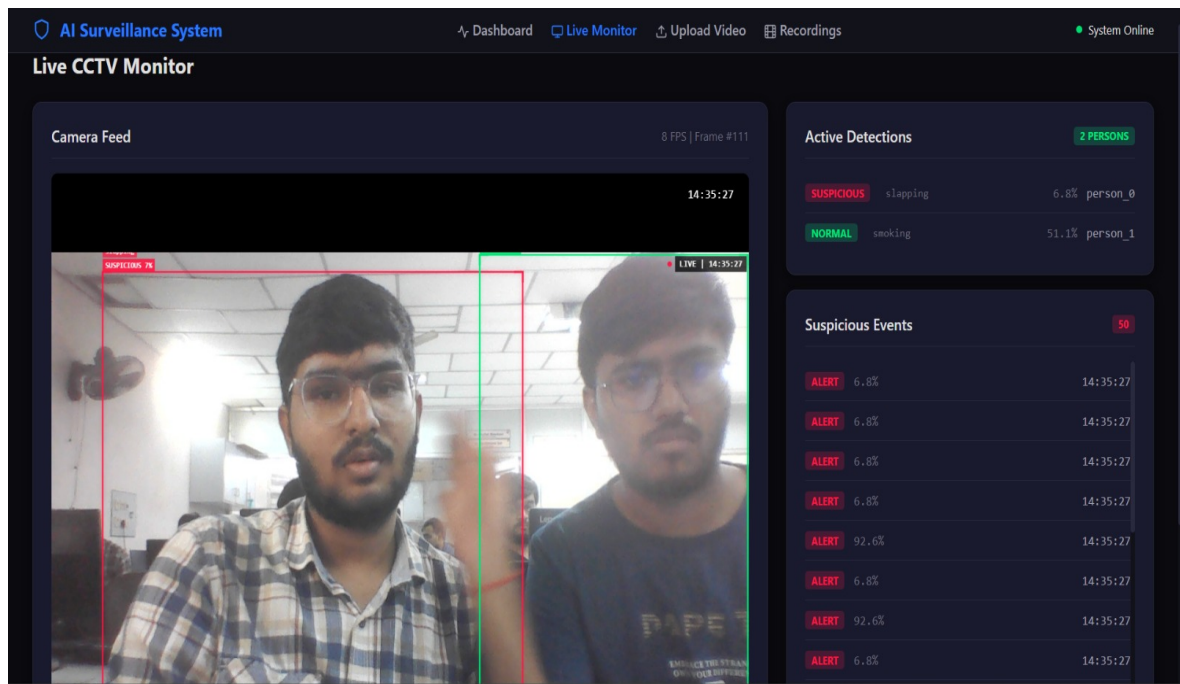


Figure 13.3: Live Monitor - Suspicious activity detection with red bounding box

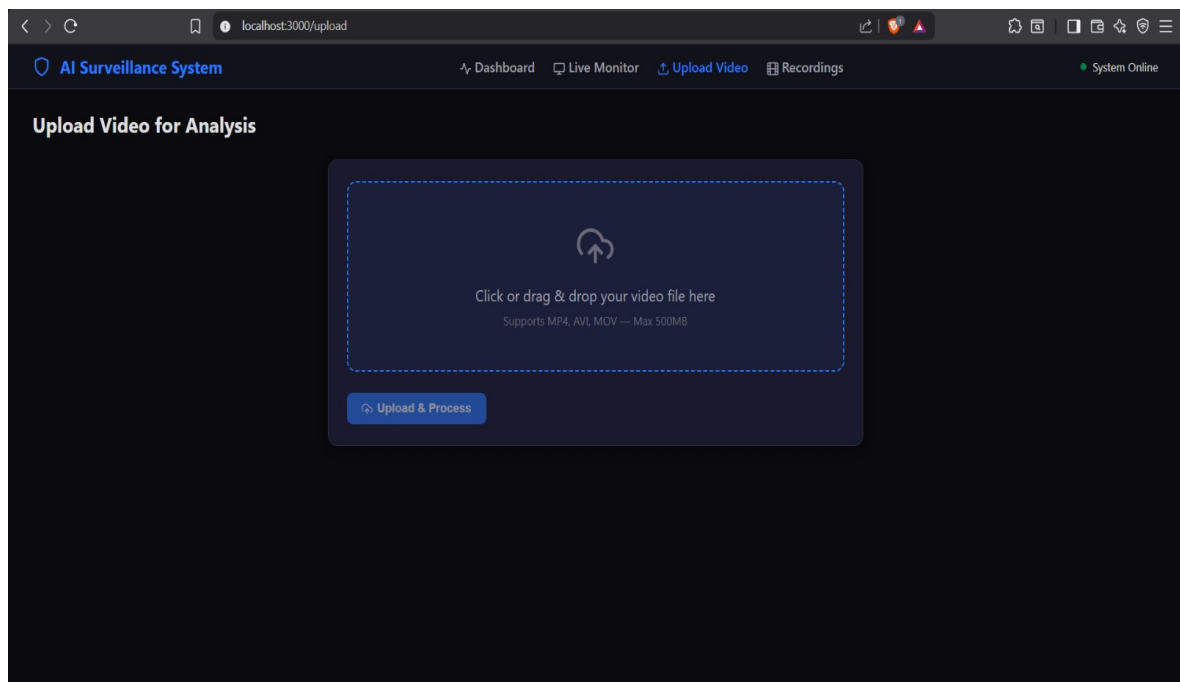


Figure 13.4: Video Upload - Drag-and-drop upload interface

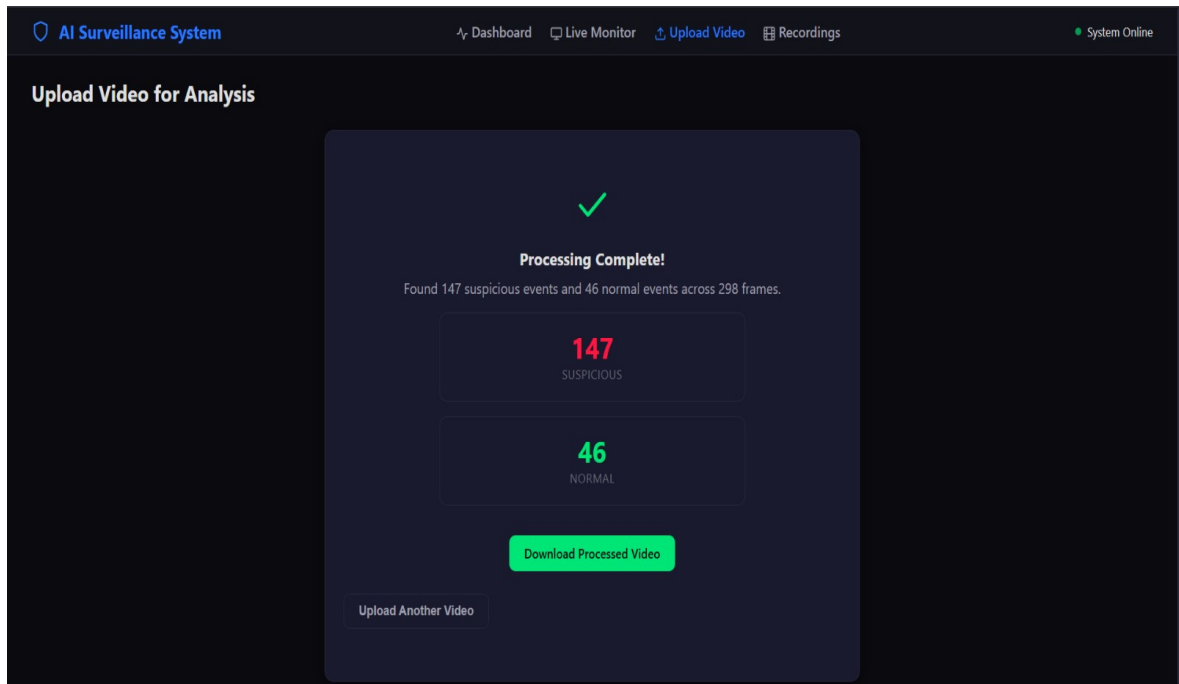


Figure 13.5: Video Upload - Processing results

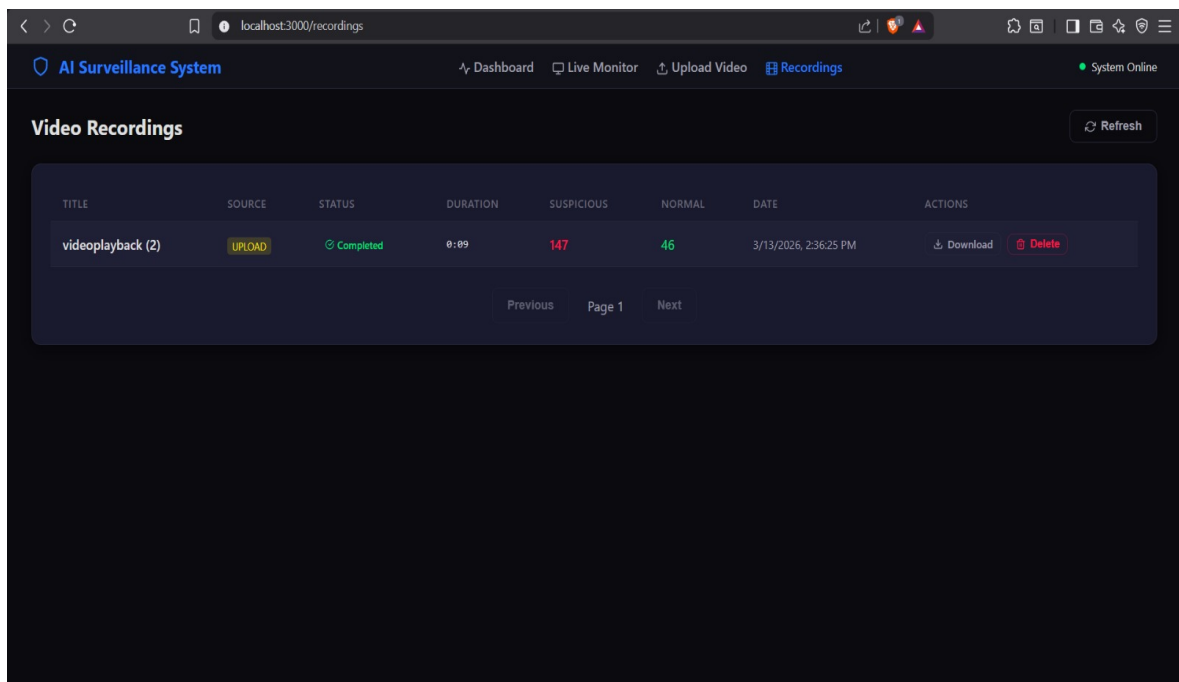


Figure 13.6: Recordings - List of recorded sessions

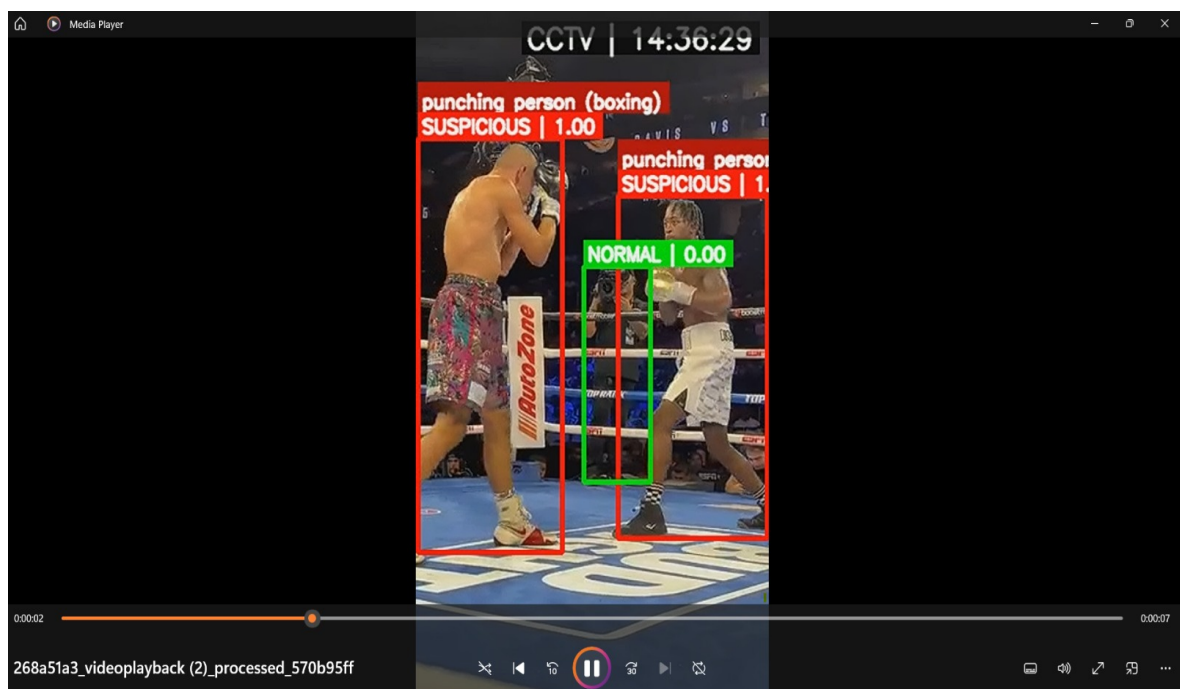


Figure 13.7: Recordings - Video playback with annotations

14. Future Enhancements

- Multi-class suspicious activity categories (e.g., vandalism, theft, assault as separate classes).
- SMS/Email alert integration for immediate notification of security personnel.
- Audio detection integration for gunshot, glass breaking, or screaming detection.
- Face recognition module for person identification and tracking across cameras.
- Edge device deployment (NVIDIA Jetson) for on-premise processing without cloud dependency.
- Multi-camera support with unified dashboard for monitoring multiple feeds simultaneously.
- Night vision / infrared camera support with model adaptation for low-light conditions.
- Historical trend analysis with weekly/monthly suspicious activity reports.
- Integration with existing security systems (alarm systems, access control).
- Mobile application for remote monitoring and push notifications.

15. Conclusion

This project successfully demonstrates the design and implementation of a Real-Time AI-Based Suspicious Activity Detection System. By combining YOLOv8 for human detection with SlowFast-R101 for temporal activity classification, the system achieves accurate and efficient real-time surveillance analysis.

Key accomplishments include:

- A fully functional end-to-end surveillance system processing video at 24 FPS.
- An optimized dual-model AI pipeline with frame skipping, classifier cooldown, and temporal smoothing.
- A tiered decision system for activity classification that minimizes false positives by restricting suspicious flags to only 11 genuinely dangerous Kinetics-400 labels.
- A responsive React frontend with live monitoring, video upload, and recording management.
- A robust Django REST API backend with comprehensive endpoints for all system operations.
- Docker-based deployment for production-ready containerization.

The system demonstrates that modern deep learning techniques, when combined with careful engineering and optimization, can provide practical, real-time surveillance solutions that augment human monitoring capabilities and improve public safety.

16. References

- [1] Feichtenhofer, C., Fan, H., Malik, J., & He, K. (2019). SlowFast Networks for Video Recognition. IEEE/CVF International Conference on Computer Vision (ICCV).
- [2] Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. GitHub repository.
- [3] Fan, H., et al. (2021). PyTorchVideo: A Deep Learning Library for Video Understanding. ACM Multimedia.
- [4] Kay, W., et al. (2017). The Kinetics Human Action Video Dataset. arXiv:1705.06950.
- [5] Django Software Foundation. (2024). Django: The Web framework for perfectionists with deadlines.
- [6] Meta Platforms, Inc. (2024). React: A JavaScript library for building user interfaces.
- [7] OpenCV Team. (2024). OpenCV: Open Source Computer Vision Library.
- [8] Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv:1804.02767.
- [9] Carreira, J., & Zisserman, A. (2017). Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset. IEEE CVPR.
- [10] Lin, T., et al. (2014). Microsoft COCO: Common Objects in Context. European Conference on Computer Vision (ECCV).